

k6 and JMeter, measured side by side at 1,000 users

Both tools were installed, configured with matched load profiles, and run against an identical target with a known response time. Every figure below was recorded on our own hardware. Nothing is quoted from a vendor.

Date 22 August 2026 **k6** v2.1.0 **JMeter** 5.6.3 · OpenJDK 17.0.18

Host Windows 11 · 8 logical cores · 15.6 GB RAM

RECOMMENDATION

Adopt k6. At our load levels both tools work correctly — so this is not a capacity argument. k6 wins on the things we do every day: tests that can be code-reviewed, thresholds that gate a pipeline, and a load generator that leaves the machine free. JMeter remains the right answer for database and message-queue protocols, which our APIs do not use.

01 How the test was run

A benchmark is only worth quoting if the two sides were given identical work. Both tools were pointed at the same purpose-built target on `127.0.0.1` and driven with matched profiles.

LOAD PROFILE

1,000 VUs

Ramp 0→1,000 over 30s, hold 60s

PER ITERATION

1 GET

Plus 1s think time

TARGET LATENCY

50 ms

Fixed delay — the known truth

SAMPLING

500 ms

RSS, threads and CPU per process

Why a fixed 50 ms target matters. Because the true server latency is known exactly, whatever each tool *reports* can be checked against reality. Any excess is the measurement instrument's own overhead. This turns a vague "which feels faster" into an arithmetic question.

TARGET — TARGET.JS

```
const server = http.createServer((req, res) => {
  setTimeout(() => { // fixed, known latency
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(body);
  }, 50);
});
```

K6 — TEST.K6.JS

```

export const options = {
  scenarios: { compare: {
    executor: 'ramping-vus', startVUs: 0,
    stages: [ { duration: '30s', target: 1000 },
              { duration: '60s', target: 1000 } ],
    } },
};
export default function () {
  const res = http.get('http://127.0.0.1:8099/api');
  check(res, { 'status 200': (r) => r.status === 200 });
  sleep(1);
}

```

JMETER — TEST.JMETER.JMX (THREAD GROUP)

```

<stringProp name="ThreadGroup.num_threads">1000</stringProp>
<stringProp name="ThreadGroup.ramp_time">30</stringProp>
<boolProp name="ThreadGroup.scheduler">true</boolProp>
<stringProp name="ThreadGroup.duration">90</stringProp>
<!-- GET /api + ResponseAssertion 200 + ConstantTimer 1000ms -->

```

No listeners were attached — listeners are the classic way to make JMeter look artificially heavy, and k6 has no equivalent cost.

02 What we measured

PEAK MEMORY

2.6×

JMeter 841 MB · k6 320 MB

OS THREADS

69×

JMeter 1,037 · k6 15

CPU CONSUMED

1.34×

JMeter 28.2s · k6 21.1s

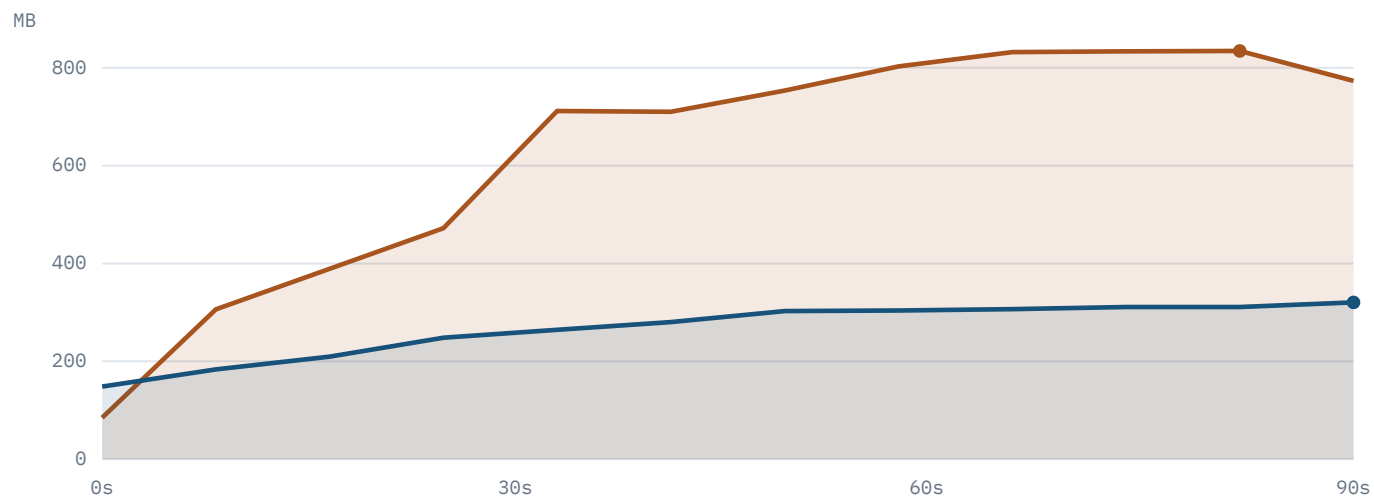
THROUGHPUT

Tie

765/s vs 784/s — both fine

METRIC	JMETER 5.6.3	K6 V2.1.0	RESULT
Peak resident memory	841.4 MB	320.5 MB	k6 uses 2.6× less
Peak OS threads	1,037	15	k6 uses 69× fewer
CPU seconds consumed	28.19	21.08	k6 uses 1.34× less
Average cores busy	0.31	0.23	Both comfortable
Requests completed	69,027	71,343	Equivalent
Throughput	765.3/s	783.5/s	Equivalent
Errors	0%	0%	Both correct
Recorded from the load-generator process every 500 ms. Both runs completed with zero errors.			

JMeter memory k6 memory



JMeter allocates its thread pool within the first 10 seconds and climbs steadily to 835 MB. k6 rises gently to 320 MB and flattens. Note JMeter's default `-Xms1g` heap accounts for part of its floor.

PEAK OS THREADS TO DRIVE 1,000 VIRTUAL USERS

JMeter – 1,037 threads

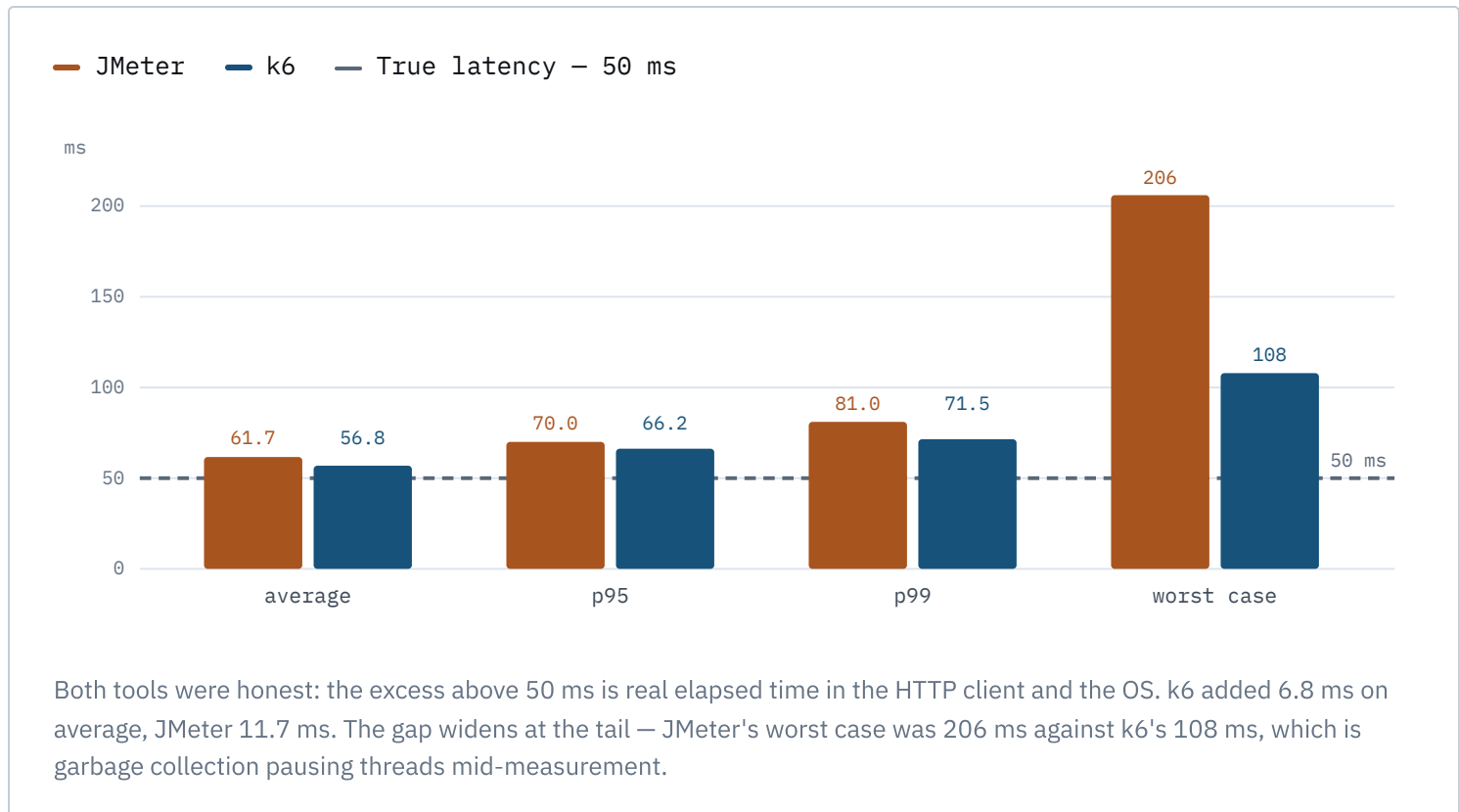
k6 – 15 threads

Bars to scale

The single most dramatic finding, and the one that follows directly from architecture: JMeter creates one operating-system thread per virtual user. k6 multiplexes all 1,000 users across a handful of threads.

03 Which tool told the truth?

The target genuinely took 50 ms. Anything above that line is overhead the measuring instrument added.



State this plainly and it will survive scrutiny: the ~5 ms difference matters for a 20 ms API and is noise for a 300 ms one. Our endpoints sit in the hundreds of milliseconds, so we are *not* claiming JMeter would give us wrong numbers. We are claiming it costs more to get the same answer.

04 Explaining it without the jargon

Waiters and diners

JMeter hires one waiter per diner. A thousand diners means a thousand waiters crowding the same kitchen — most stood still, waiting on food, while the kitchen jams with bodies.

k6 employs fifteen waiters who never stand still: place an order, move to the next table, come back when the food is up.

Same meals served. 1,037 staff versus 15.

The stopwatch that takes time to click

Every measuring instrument disturbs what it measures. Timing a 50 ms request with a stopwatch that itself takes 12 ms to press gives a different answer than one that takes 7 ms.

Neither lied. One is a finer instrument.

Blueprint versus assembly notes

A JMeter test is 800+ lines of machine-written XML. Nobody can review a change to it — you approve it because it ran, not because you read it.

A k6 test is 50 lines of JavaScript that a colleague reads in a pull request like any other code.

Tests we cannot review are tests we cannot trust.

The queue that measures itself

Time a restaurant queue by only clocking people who got seated, and you miss everyone who gave up and left. Load tools do this when they wait for a slow reply before sending the next request.

Both tools have it. Both can fix it — k6 with arrival-rate executors, JMeter with the Open Model Thread Group. This is a configuration choice, not a tool choice.

05 Where JMeter is the better tool

A recommendation that only lists one side's strengths invites the question of what was left out. These are real and we checked them.

- **Protocol coverage.** JMeter tests JDBC, JMS, LDAP, FTP and SMTP natively. k6 cannot. If we ever load-test a database or message queue directly, JMeter is the answer.
- **Free distributed testing.** JMeter's controller/worker mode is built in and runs on plain VMs. k6's equivalent needs a Kubernetes cluster or paid Grafana Cloud.
- **Record and replay.** JMeter's proxy recorder captures a browser session into a test plan. k6 has no equivalent.
- **Non-programmers.** JMeter's GUI is genuinely the gentler start for someone who will never write code.
- **API stability.** k6 v2 removed the externally-controlled executor, which cost us live VU scaling. A 20-year-old Apache project does not do that.
- **Community depth.** Two decades of JMeter answers versus a thinner k6 corpus.

Why none of these change the decision. Our APIs are HTTP and JSON, so the protocol gap never opens. At 300–500 concurrent users a single machine is ample, so distributed mode never opens. Our team reads JavaScript daily. Of k6's drawbacks, only the version-stability risk can reach us — and pinning the k6 version in the repository closes it.

06

Scored against our actual work

WHAT WE DO	BETTER TOOL	WHY IT MATTERS HERE
HTTP / JSON API testing	k6	Both capable; k6 is lighter and quicker to author
Tests reviewed in pull requests	k6	Readable JavaScript instead of generated XML
Pipeline pass/fail gates	k6	Thresholds and exit codes are built in
One machine, 300–500 users	Either	Measured: both idled at ~0.3 of 8 cores
Running alongside other work	k6	320 MB and 15 threads leaves the laptop usable
Onboarding a non-coder	JMeter	Mitigated: our runner takes a Postman collection
Database / queue protocols	JMeter	Not applicable to our stack today
Weighted by how often each factor appears in our week, not by how impressive it sounds.		

DECISION

Standardise on k6 for API load testing. Keep JMeter in mind for the day we need a protocol k6 does not speak — adding it for that one job costs far less than migrating everything now. The two are not mutually exclusive, and plenty of teams run both.

07 Limits of this benchmark

Declaring these up front is what separates a measurement from a marketing claim — and it is the first thing a sceptical reviewer will probe.

- **JMeter pre-allocates a 1 GB heap** (-Xms1g -Xmx1g by default). Part of its 841 MB is that configured floor, not per-thread cost. Tuning the heap would narrow the memory gap.
- **JMeter wrote a 7.4 MB results file**; k6 wrote only a summary. That is real I/O k6 did not pay for. It is standard JMeter practice, but not a like-for-like engine comparison.
- **Generator and target shared one 8-core machine.** Neither was starved — both used about a third of one core — but this is not an isolated lab.
- **1,000 users is not where JMeter struggles.** The divergence grows at 5,000–10,000+, where thousands of OS threads contend for a handful of cores. We did not test that, and we do not claim it.
- **A single run each.** Repeat runs would tighten confidence in the CPU figure especially.

Independent corroboration. A separately published comparison recorded JMeter at 760 MB against k6 at 256 MB. Our run gave 841 MB against 320 MB — different hardware, the same ratio. The memory finding is not an artefact of our setup.

08 Sources

Primary data is our own. The following corroborate the architectural claims and version history.

[k6 — Running large tests \(official documentation: 1–5 MB per VU, 30,000–40,000 VUs per machine\)](#)

[Apache JMeter — Best Practices \(official: non-GUI mode, thread sizing, coordinated omission\)](#)

[Apache JMeter releases \(latest stable 5.6.3, January 2024\)](#)

[Grafana k6 releases \(v2.x line; v2.0 removed the externally-controlled executor\)](#)

[BlazeMeter — Maximum concurrent users in JMeter \(200–300 threads per engine guidance\)](#)

[Exploring the impact of stack size on JVM thread creation — a myth debunked](#)

[ScyllaDB — On coordinated omission](#)

[Red Hat Performance — Coordinated omission](#)

[Distributed load testing on Kubernetes: JMeter, k6 and Gatling](#)

[k6 vs JMeter benchmarked on a real app \(self-described as directional, not lab-grade\)](#)

Prepared from measurements taken on 22 August 2026. Test harness — `target.js`, `test.k6.js`, `test.jmeter.jmx`, `measure.ps1` — is reproducible and available with this report.